MatchMyStuff

Powered by AI. Driven by kindness.

AI-powered lost-and-found platform that connects people who lost items with people who found them

using computer vision · semantic embeddings · real-time notifications · in-app chat

Track	Team	Stack
Convex	**Team CrackHeadz**	**Next.js + Convex + OpenAI**

01 Project Overview

Project Name & One-Line Pitch

Project Name: MatchMyStuff

Tagline: Powered by AI. Driven by kindness.

One-liner: An AI-powered lost-and-found web application that automatically matches lost items with found items using computer vision and semantic embeddings, enabling matched users to coordinate returns through real-time in-app chat.

Summary

Losing personal belongings is a stressful and frustrating experience, and existing recovery channels — Facebook groups, WhatsApp chats, physical notice boards, and police desks — are fragmented, slow, and rely entirely on manual browsing. MatchMyStuff solves this by providing a single structured platform where lost and found reports are automatically matched by AI.

Users report a lost or found item with a photo and description. The backend validates the photo using GPT-4o vision, generates an AI description, creates a 1536-dimensional semantic embedding, and searches for opposite-type posts using Convex vector search. When a match scores above 82%, both users are notified instantly and can communicate through a match-gated chat system without ever exposing personal contact details publicly.

The platform is built for urban residents in Sri Lanka — students, commuters, and community members — who want a faster, smarter, and more trustworthy way to recover lost belongings or return found items.

Submission Details

Track	Convex — The track technology (Convex) is meaningfully integrated as the entire backend: database, real-time queries, file storage, auth, vector search, and serverless actions all run on Convex.
Demo URL	localhost:3000 (live demo available during presentation)
Repository	Available upon request
Core AI	OpenAI GPT-4o (vision validation + description) + text-embedding-3-small (embeddings)

02 Problem Statement

The Problem

People lose personal belongings — phones, wallets, keys, bags, headphones — every day in urban environments. When this happens, they have no reliable digital channel to report the loss and be automatically alerted when someone finds it. Finders face the same friction: they want to return the item but have no private, structured way to do so.

The consequences of this gap are real: items go unreturned not because no one found them, but because the finder and owner never connected. The cost falls hardest on people who cannot easily replace what they lost.

Why It Matters

- High daily frequency: wallets, phones, and keys are among the most commonly lost items in any city, with thousands of incidents daily across Sri Lanka's urban centres.
- Workarounds fall short: Facebook groups require manual searching; WhatsApp requires knowing the right group; police desks are slow and underfunded; physical boards are unstructured and weather-dependent.
- Keyword mismatch: a person who lost a "pink Nike sneaker" will never find a post titled "running shoe" — pure text search fails because people describe the same item differently.
- Privacy risk: posting a found item publicly with contact details exposes the finder to spam and the item to false claimants.
- No proactive matching: every existing channel requires the owner to actively search — they are never alerted when a matching item appears.

Root Cause

The root cause is the absence of a structured, semantically aware matching layer. Existing platforms treat lost-and-found as a search problem, but it is fundamentally a matching problem — and matching requires understanding what an item is, not just what words were used to describe it. No existing tool in this market applies multimodal AI to bridge that gap.

03 Proposed Solution

What the Product Does

MatchMyStuff is a web platform where users report lost or found items with a structured form — title, description, location, and photo. Once submitted, an automated AI pipeline validates the photo (rejecting selfies, memes, and irrelevant images), generates a detailed item description using GPT-4o vision, combines it with the user's own description, and produces a 1536-dimensional semantic embedding using OpenAI's text-embedding-3-small model.

The embedding is stored in Convex and immediately searched against all opposite-type posts (lost ↔ found) using Convex's built-in vector index. A hybrid score — 90% semantic similarity plus 10% location match — determines whether a match is strong enough. When the score reaches 82% or above, both users are notified in real time, their posts are flagged as matched, and a private chat conversation is created so they can coordinate the return without ever sharing personal contact details publicly.

Key Features

Feature	User Need Addressed
AI image validation	Rejects low-quality or irrelevant photos before they enter the feed, keeping listings trustworthy
GPT-4o vision description	Understands what the item actually is, not just what words the user chose
Semantic embedding + vector search	Matches items even when described differently; "pink sneaker" matches "running shoe"
Hybrid scoring (similarity + location)	Reduces false matches by weighting geographic proximity alongside semantic match
Real-time in-app notifications	Users are alerted instantly — no need to manually check the feed
Match-gated private chat (text/image/location)	Finders and owners communicate privately without exposing personal contact details
Public map with geocoded pins	Community members can browse and help even without creating a post
Admin moderation panel	Platform stays trustworthy through human oversight of posts, users, and matches
Processing status lifecycle	Users know exactly where their post is in the pipeline (pending → processing → ready/rejected)

Scope

In scope for this build:

- Full lost/found reporting with photo upload and describe-only mode for lost items
- Complete AI pipeline: validation, description, embedding, matching, notification, email
- Real-time in-app chat with text, image, and GPS location sharing
- Public feed with search, Lost/Found tabs, and interactive map
- Admin dashboard for content moderation
- Email notification via Gmail on match

Deliberately out of scope:

- Mobile native app (web-only)
- Semantic/vector-based feed search (keyword only in this build)
- Payment, escrow, or proof-of-return workflow
- Institutional/B2B portal customisation

04 Functional Requirements

Must Have — demonstrable in live demo

- User can sign up and sign in with email and password via Convex Auth
- User can report a lost item with optional photo or description only
- User can report a found item with a required photo
- System validates uploaded photo via GPT-4o before listing goes live
- System generates AI description and semantic embedding for each valid post
- System automatically searches for opposite-type matches after embedding
- Match created when hybrid score >= 0.82; both posts marked matched
- Both users receive in-app notification on match
- Users can view match details with confidence score and other post
- Matched users can open a private chat and send text messages
- Public homepage feed shows recent posts with Lost/Found tab filter
- Protected routes redirect unauthenticated users to /auth

Should Have

- Email notification via Gmail on match
- Chat supports image sharing and GPS location sharing
- Map view with geocoded pins for all ready posts
- Post processing status visible to owner (pending / processing / ready / rejected)
- Admin panel with post, user, and match management
- Keyword search across title and description in the feed
- My Posts page showing user's own submissions and their status

Could Have / Won't Have This Build

- Semantic/vector feed search — keyword only due to time constraints
- Mobile native app — web only
- Reward or escrow system for successful returns
- Multi-language support — English only
- Institutional/branded portal for universities or transport operators

05 Non-Functional Requirements

Performance

- AI pipeline runs asynchronously via Convex scheduler — post creation returns immediately without waiting for GPT-4o or embedding
- Feed search is debounced at 400ms to avoid excessive query calls
- Map geocoding is batched with delay and cached in-memory on the API route to respect Nominatim rate limits
- MapView dynamically imported with no SSR to prevent hydration issues

Reliability & Error Handling

- Posts that fail AI validation receive a rejectionReason and a clear error state on the report UI — they never enter the public feed
- Email notification is skipped gracefully if Gmail credentials are not configured; a warning is logged but the match still creates
- Geocode API tries Photon first and falls back to Nominatim automatically
- Embedding search only runs on posts that have reached ready status with a non-empty embedding array

Usability

- Responsive layout throughout; mobile devices hide hero decorative elements to preserve performance
- All form fields include accessible labels and inline validation errors in brand coral
- Processing status is surfaced clearly so users are never left wondering why their post is not yet live
- Toast feedback confirms successful actions (post submitted, message sent, match marked seen)
- Brand colours (teal, sky, coral, slate) are centralised in lib/colors.ts; all copy in lib/copy.ts

Scalability

- Convex serverless backend scales automatically — no manual server provisioning
- Vector index on posts table allows efficient nearest-neighbour search at scale without a separate vector database
- Async processing via ctx.scheduler.runAfter decouples user-facing mutations from expensive AI calls
- For higher load: add per-user rate limiting on OpenAI calls; introduce geocode caching in Convex rather than in-memory

06 Technical Architecture

System Overview

MatchMyStuff runs as two processes: npx convex dev serves the Convex backend (database, auth, file storage, serverless actions, HTTP routes), and npm run dev serves the Next.js 16 frontend at localhost:3000. The browser communicates with Convex via ConvexReactClient wrapped in ConvexAuthNextjsProvider. All AI calls originate from Convex actions (Node runtime) — never from the Next.js frontend — keeping the OpenAI API key server-side only.

Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 16 (App Router), React 19, TypeScript	Server components, file-based routing, full TypeScript safety
Styling	Tailwind CSS v4 + Framer Motion	Utility-first styling; smooth animation without a library
Backend / DB	Convex	Real-time queries, serverless mutations, built-in vector search, file storage, and auth in one platform
Auth	@convex-dev/auth (Password + Anonymous)	Native Convex auth — no third-party service needed; server-side getAuthUserId on every mutation
AI - Vision	OpenAI GPT-4o	Best-in-class multimodal model for image validation and item description
AI - Embeddings	OpenAI text-embedding-3-small	1536-dim embeddings at low cost; sufficient quality for item similarity matching
Maps	react-leaflet + Leaflet + OSM	Open-source; no API key required; Sri Lanka tile coverage
Geocoding	Photon API + Nominatim fallback	Free, open-source; Photon is fast; Nominatim as reliable fallback
Email	Nodemailer + Gmail SMTP	Zero-dependency email for match notifications
Hosting	Convex cloud + Vercel (Next.js)	Both platforms have generous free tiers suitable for hackathon deployment

Data Flow — Core Action (Report a Found Item)

1. User fills report form at /report/found — title, description, location, photo
2. Browser calls generateUploadUrl mutation (auth required) → receives Convex storage upload URL
3. Browser PUTs file directly to Convex storage URL → receives storageId

4. Browser calls createPost mutation with all fields + storageId
5. createPost resolves storageId → imageUrl, inserts post (matched: false, embedding: []), schedules processPost action
6. processPost (Node action): loads post, calls GPT-4o with imageUrl → aiDescription
7. processPost: concatenates title + description + aiDescription + location → calls text-embedding-3-small → 1536-dim embedding
8. processPost: calls updateEmbedding internal mutation → patches post with embedding + aiDescription
9. processPost: schedules findMatches action
10. findMatches: calls ctx.vectorSearch on posts / by_embedding, filters opposite type (lost), limit 20
11. findMatches: for each candidate, loads full post, computes cosine similarity + location score → finalScore
12. If finalScore >= 0.82: calls createMatch → inserts match, creates 2 notifications, patches both posts matched: true, sends email
13. Both users see bell count update in real time via Convex useQuery subscription

AI Integration

Validation model	GPT-4o — receives the uploaded image URL and a fixed prompt instructing it to confirm the image shows a real physical item. Rejects selfies, memes, screenshots, and non-item images before the post enters the feed. Returns structured JSON with isValid and rejectionReason.
Description model	GPT-4o vision — receives the image and generates a concise (~80 word) item description covering object type, colour, brand, condition, and distinguishing features for use in embedding.
Embedding model	text-embedding-3-small — receives combined text (title + description + aiDescription + location) and returns a 1536-dimensional float vector stored in Convex alongside the post.
Vector search	Convex vectorSearch on the by_embedding index, filtered to opposite type. Returns up to 20 candidates ranked by cosine similarity.
Scoring	finalScore = cosineSimilarity × 0.9 + locationScore × 0.1. Location score: 1.0 exact match, 0.5 substring, 0 otherwise. Threshold: 0.82.

Known Technical Limitations

- Feed search is keyword-only — it matches tokens in title and description but is not semantic; two posts describing the same item differently may not surface together in search (though they will still be matched by the AI pipeline).
- Homepage statistics (2,400+ recovered, 98% accuracy, < 2 min) are aspirational marketing copy, not computed from live data.
- Admin authentication uses a plain-text password comparison against an environment variable — not cryptographically hashed.
- Admin session token is stored in localStorage, which is vulnerable to XSS if any such vulnerability exists.

- No per-user rate limiting on OpenAI calls — a single user could trigger many expensive AI requests by submitting posts rapidly.
- Geocode API route (/api/geocode) is unauthenticated and could be abused for scraping.

07 Security

Authentication & Authorisation

- Users authenticate via @convex-dev/auth Password provider — email and password; sessions are managed by Convex Auth with JWT and JWKS.
- Every mutation and query that requires authentication calls getAuthUserId(ctx) server-side — the userId is never trusted from the client.
- Next.js middleware (convexAuthNextjsMiddleware) protects /report/*, /matches/*, /my-posts/*, /conversations/*, and /chat/* — unauthenticated users are redirected to /auth.
- Admin panel uses a separate session table and environment credentials (ADMIN_EMAIL, ADMIN_PASSWORD). An adminToken is required on every admin mutation and query.
- Chat is match-gated: getOrCreateConversation verifies the requesting user owns one of the matched posts before creating or returning a conversation.

Data Handling

- All user data (posts, matches, notifications, messages) is stored in Convex — a SOC 2 compliant platform.
- Images are stored in Convex file storage and served via signed temporary URLs — not publicly addressable by default.
- Contact emails of matched parties are only exposed within the authenticated matches view, not on public post cards.
- Found-item map popups show only a general area for non-owners and non-matched users — precise location is withheld until a match is established.

API & Secret Management

- OPENAI_API_KEY is set as a Convex deployment environment variable and accessed only from Convex actions (Node runtime) — never exposed to the browser.
- GMAIL_USER and GMAIL_PASS are Convex environment variables — never in the Next.js frontend or repository.
- NEXT_PUBLIC_CONVEX_URL is the only public environment variable; it exposes the Convex deployment endpoint (by design).
- No API keys or secrets appear in the repository codebase.

Input Validation

- Form fields enforce minimum lengths (title: 3 chars, description: 10 chars) client-side with inline errors.
- Image uploads pass through GPT-4o validation before a post is marked ready — invalid images trigger a rejection with a clear reason.
- Convex schema enforces types on all fields — malformed inputs are rejected at the database layer.

Known Vulnerabilities or Gaps

- Admin password is compared as a plain string against an env variable — not bcrypt-hashed. Acceptable for a hackathon demo; would require hashing in production.
- Admin token in localStorage is vulnerable to XSS — would move to httpOnly cookie in production.
- Geocode API route is unauthenticated — would add rate limiting or require auth in production.
- No explicit rate limiting on AI calls per user — would add per-user throttling in production.

08 User Stories & Use Cases

Core User Stories

- **Lost item owner:** I want to report my lost item with a photo so that the AI can understand what it looks like and find a match automatically.
- **Lost item owner:** I want to receive an instant notification when a found post matches mine so that I do not have to keep checking the platform.
- **Finder:** I want to report a found item with a photo so that the legitimate owner can be identified and contacted without me posting my personal details publicly.
- **Finder:** I want to chat privately with the matched owner so that we can arrange a handoff without sharing phone numbers in public.
- **Either user:** I want to see the match confidence percentage so that I can judge how likely this is the right item before starting a conversation.
- **Either user:** I want to share my GPS location in chat so that we can agree on a meetup point without leaving the app.
- **Visitor:** I want to browse the public feed and map so that I can check whether my item was reported or help return something I found.
- **Admin:** I want to delete fraudulent posts and users so that the platform remains trustworthy for the community.

Primary Use Case Walkthrough — UC4: Automatic AI Match

14. User A signs in and reports a lost black backpack at Colombo Fort station, uploading a photo.
15. createPost inserts the post and schedules processPost asynchronously.
16. GPT-4o validates the photo (confirms it shows a real item) and generates: "Black Herschel backpack with orange zipper pulls, medium size, worn condition."
17. text-embedding-3-small embeds the combined text → 1536-dim vector stored in Convex.
18. findMatches runs ctx.vectorSearch filtering to found posts, limit 20.
19. User B had earlier reported a found black backpack at Colombo Fort — their embedding is returned with cosine similarity 0.91.
20. Location score = 1.0 (exact match). finalScore = 0.91 × 0.9 + 1.0 × 0.1 = 0.919.
21. Score >= 0.82: createMatch inserts a match record, creates two notifications, marks both posts matched.
22. User A and User B both see their notification bell count increment in real time.
23. User A opens the notification, views the match detail at /matches/[id] with 91% confidence.
24. User A clicks "Open Chat" → getOrCreateConversation creates a conversation tied to the match.
25. Both users exchange messages, User B shares GPS location → they arrange to meet.

Edge Cases

- Photo of a selfie or meme submitted as a found item → GPT-4o validation rejects it; post gets status rejected with rejectionReason shown to user; post never enters feed or matching.

- Two lost posts with identical descriptions → matching only runs lost ↔ found; lost–lost pairs are never compared.
- Score below threshold (e.g. 0.75) → no match created; both posts remain active and will be re-compared if new posts arrive.
- User submits a post without a photo (lost, describe-only) → embedding is built from title + description + location only; matching still runs with lower confidence.
- Match email fails (Gmail credentials missing) → match is still created; notification still appears in-app; only email is skipped.
- User tries to access /chat/[id] without being a conversation participant → getConversation returns null; page shows access denied.

09 Target Users & Market

Primary User

Urban residents in Sri Lanka — primarily students, commuters, café patrons, and transport users — who have lost or found a personal belonging and want a faster, more reliable recovery channel than Facebook groups or WhatsApp. The specific pain: they currently have no way to be automatically alerted when a matching item appears, and no private coordination channel once a potential match is identified.

Market Opportunity

- Sri Lanka has a population of ~22 million, with dense urban centres in Colombo, Kandy, and Galle where daily commuting and public-space activity creates a high volume of lost-item incidents.
- The informal lost-and-found market (Facebook groups, WhatsApp, police desks) is entirely undigitised and unstructured — no incumbent with AI matching exists in this segment.
- Secondary opportunity: institutional buyers — universities, shopping malls, ride-share companies, hotels — who want a branded lost-and-found portal without building one.

Competitive Landscape

Alternative	What It Does	Key Weakness
Facebook Groups	Community members post photos and descriptions in local groups	No matching; requires manual scrolling; contact details are public; no privacy
WhatsApp Chains	Photos shared in neighbourhood or university groups	No structure; depends on being in the right group; no notification when match appears
Police Lost Property	Formal registration of lost items	Slow; no digital search; no photo matching; no proactive notification

010 Business Model

Revenue Model

MatchMyStuff launches as a free platform to build the network density required for matching to work — more posts means better matches for everyone. Once volume is established, three monetisation paths are viable:

- **Freemium:** Free basic reporting; paid priority matching (post processed first in queue) or boosted visibility on the public feed.
- **B2B / Institutional:** License the platform to universities, shopping malls, transport operators, and hotels as a branded lost-and-found portal. Recurring SaaS fee per organisation.
- **Civic / NGO grants:** Apply for social impact or civic-tech funding given the community benefit in a developing market context.

Pricing Hypothesis

- Individual users: free tier permanently; premium at ~LKR 500/month (~$1.50) for priority processing and extended match history.
- Institutional: LKR 15,000–50,000/month (~$50–$170) depending on post volume and customisation — comparable to basic SaaS tools already in use by Sri Lankan universities.

Go-to-Market

- First 10 users: direct outreach to university student unions in Colombo (UoM, SLIIT, NSBM) — students lose items in campus cafeterias, labs, and transport daily.
- Channel: Instagram and LinkedIn posts showing a real match story ("this wallet was returned in 6 minutes") — emotional, shareable, and demonstrates the product.
- Partner with one university to run a pilot — gives a real dataset, testimonials, and a reference customer for B2B sales.

Roadmap

Now (built)	Full MVP: reporting, AI pipeline, matching, notifications, chat, map, admin panel
0–3 months	Semantic feed search; push notifications (PWA); mobile-optimised UI; pilot with one university
3–12 months	Institutional portal product; verified account badges; reward/escrow on successful return; expand to additional cities
1 year+	Multi-country expansion (South/Southeast Asia); API for third-party integrations; native mobile app

011 Why This Project Leads the Track

Technical Edge

- Full multimodal AI pipeline: GPT-4o validates images and generates descriptions; text-embedding-3-small produces semantic vectors; Convex vector search finds nearest neighbours — all automated, all server-side, all within a hackathon timeframe.
- Convex is used end-to-end — not just as a database. It handles auth, file storage, real-time subscriptions, serverless actions (Node runtime for OpenAI), scheduled jobs (ctx.scheduler.runAfter), and vector search. No secondary services required.
- Hybrid scoring (cosine similarity × 0.9 + location score × 0.1) is a deliberate algorithmic choice — not just a threshold query. It reflects real-world matching logic where two items that look identical but are in different cities are less likely to be the same item.
- Match-gated chat with three message types (text, image, GPS location) is a complete coordination layer — not a proof of concept. It includes seen indicators, date groupings, image lightbox, and reverse geocoding via Nominatim.
- Processing lifecycle (pending → processing → ready | rejected) means the system fails gracefully and communicates failure clearly — a sign of production thinking, not just demo polish.

Problem-Solution Fit

The problem — lost items go unreturned because finder and owner never connect — is real, frequent, and felt personally by almost everyone. The solution addresses the root cause directly: semantic understanding of items, not keyword search. Every architectural decision traces back to this: vision for understanding, embeddings for matching despite different descriptions, private chat for trust, photo validation for quality. Nothing in the architecture is decorative.

Execution Quality

- 17 pages / routes built and functional: /, /auth, /report/lost, /report/found, /post/[id], /matches, /matches/[id], /my-posts, /conversations, /chat/[id], /admin/*, /terms, /privacy, /contact.
- Complete admin panel with post, user, and match moderation — a feature most hackathon projects skip.
- Framer Motion animations, brand-consistent design system (lib/colors.ts, lib/copy.ts), responsive layout, and toast feedback — polished enough to demo to real users today.
- Real-time updates throughout: notification count, feed, chat — all via Convex useQuery subscriptions with zero polling.

Real-World Potential

MatchMyStuff is not a toy. It solves a problem that exists right now, in Sri Lanka, every day. The AI pipeline is production-quality (GPT-4o + embeddings at scale is exactly how real semantic search systems are built). The network-effect dynamic — more posts means better matches — is a legitimate growth mechanic. One successful, emotional match story shared on social media is enough to seed organic growth. The institutional B2B path (universities, malls, transport) gives a clear revenue route without depending on consumer scale.

012 Team & Roles

Team Members

Name	Role	What They Built	Background
Sentheepan	Project Lead & Full-Stack	Frontend UI, landing page, post creation forms, match viewer, real-time notifications UI	Full-stack developer with React and Next.js experience
Kishgintharaam	Backend Engineer	Convex schema, all mutations/queries, auth setup, middleware, provider wiring, integration of all features	Backend and database engineering; Convex specialist for the team
Kajanthan	AI & Pipeline Engineer	OpenAI actions (vision + embeddings), findMatches logic, hybrid scoring, image upload pipeline, Vercel deploy	AI/ML integration and API engineering

Why This Team

The team combines three complementary skill sets — frontend/UX, backend/infrastructure, and AI/ML integration — with no overlap and no gaps for this specific build. The division of labour meant all three tracks (UI, Convex backend, AI pipeline) could develop in parallel, with a dedicated integration phase where the backend engineer connected the frontend components to real Convex queries and wired the AI actions into the post lifecycle. The result is a product where every layer was built by someone who understood it deeply, rather than a single developer stretched across the entire stack.